AOSP 进阶丛书 · 第三册 · ANDROID 13 (API 33)

Binder IPC 全链路剖析

跟随 App 调用 AMS 的一个方法穿过十跳：mmap 一次拷贝原理、BC/BR 命令协议、servicemanager 名录与门禁、AIDL 生成物解剖、线程池与死亡通知——跨进程调用从此白盒。

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

目 录

前言：一个调用打穿整个系统 2
第一章 为什么是 Binder 2
第二章 总体架构：五层纵切 4
第三章 驱动协议：BC/BR 与核心结构 5
第四章 ProcessState：进程的 Binder 资产 8
第五章 IPCThreadState：命令收发中枢 8
第六章 Parcel：跨进程的类型系统 11
第七章 servicemanager：名录与门卫 12
第八章 Java 层与 AIDL：对象的两个世界 13
第九章 端到端全链路：把图走成路 19
第十章 线程池与高级机制 20
第十一章 设计决策十二问 23
第十二章 深水区：Binder 观测武器库与 RPC 后端 23
附录 A 观测实战 24
附录 B 三册联动：一条完整的因果链 25
附录 C 自检题 25
附录 D 术语速查 25
附录 E 动手实验（一天量） 26
教科书习题集（分级） 26
术语表（按拼音排序） 27
排障手册：Binder 问题定位 29

本册速览

一次拷贝的 mmap 原理与 1MB-2 页账本 · BC/BR 命令协议
端到端十跳（App→驱动→AMS） · 服务注册五道关卡 · AIDL 生成物解剖
线程池 15+1/31 · 死亡通知 · oneway 真实语义 · 冻结 BR_FROZEN_REPLY
深水区：三层观测武器库与 RPC 后端
先读第一章对比表再入十跳；附录 E 的 MiniBinder 动手做一遍

前言：一个调用打穿整个系统

第一册说“一切跨层皆 Binder”，第二册让 system_server 从 zygote 里站了起来——本册补上最后一块：**进程们如何对话**。你将跟随 App 调用 AMS 的一个方法，从 Java 门面走进 JNI、穿过 libbinder、越过内核驱动、落到服务端线程栈，再原路返回。读完本册，“跨进程调用”对你将不再是黑盒，而是十跳可以逐一命名的白盒。

源码基线同前两册（AOSP Android 13）。特别说明：本树 `kernel/` 不含驱动源码（binder.c 在独立内核仓），驱动侧内容依据稳定 UAPI 头 `external/kernel-headers/original/uapi/linux/android/binder.h` 与 libbinder 的调用序列反推，图会明确标注这一点。

第一章 为什么是 Binder

> **设计洞察 • 学习目标**
> 1. 对比四种IPC的账单；2. 解释一次拷贝的原理；3. 画出mmap映射图

1.1 四种 IPC 的对账单

机制	拷贝次数	安全模型	同步语义	能力传递
管道/Unix socket	2 次	UID 自报（可伪造为 root 视角）	阻塞读写	无
SysV/POSIX shm	0 次	密钥约定，无内核仲裁	无（自建）	无
信号量/消息队列	1-2 次	IPC 权限位	弱	无
Binder	**1 次**	**内核采集 euid/pid + SELinux 上下文**	同步/oneway/死亡通知/引用传递	**对象能力（binder node/ref）**

Binder 从不是“最快的”（共享内存零拷贝），但它是**每微秒买到最多语义**的：一次拷贝顺带换来了能力模型（跨进程的对象引用）、内核背书的身份、死亡通知与有序投递。Android 的一切权限检查都建立在这张账单上。

1.2 一次拷贝是怎么做到的

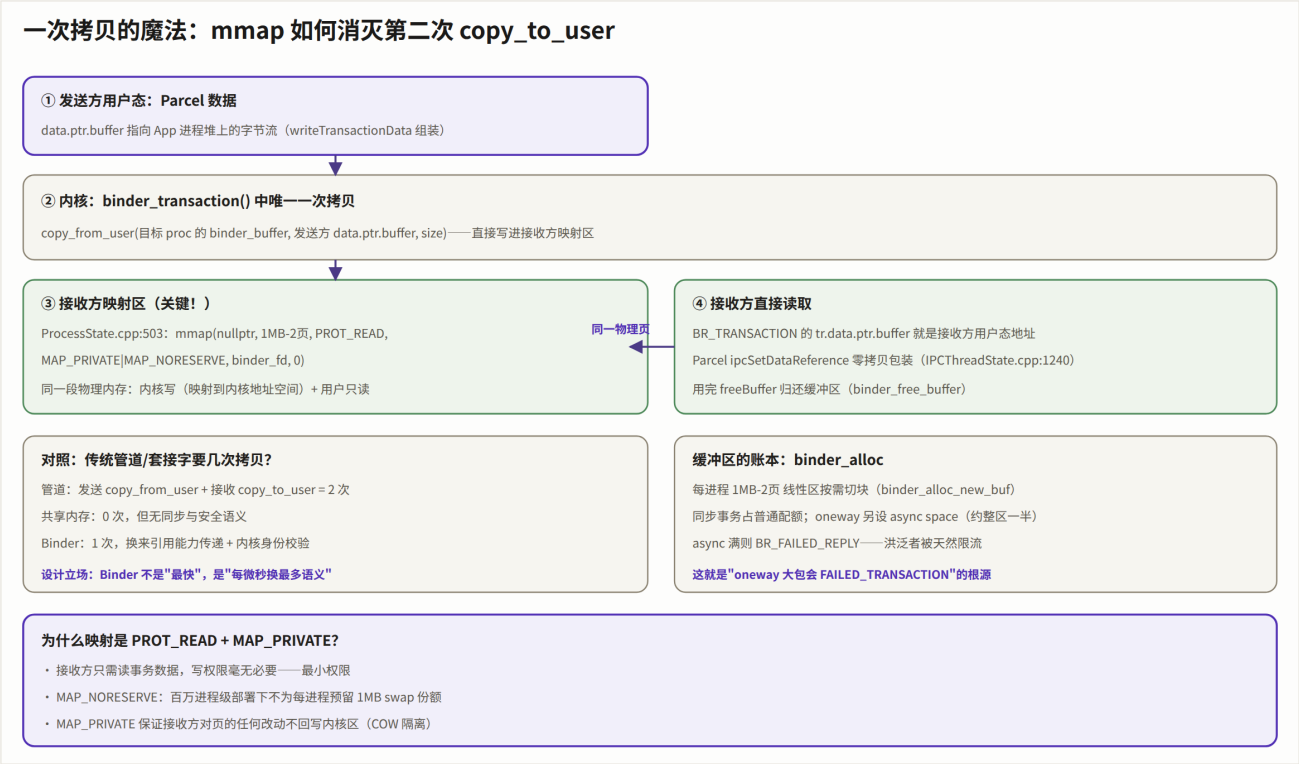


图 1 mmap 一次拷贝示意图：接收方 1MB-2 页只读映射区、copy_from_user 直写、缓冲账本与三条 mmap 参数的设计动机

关键在于 **mmap 的方向**：接收进程对 `/dev/binder` 调 mmap（`ProcessState.cpp:503`）后，内核把该进程的接收缓冲区同时映射进内核地址空间与该进程用户态（只读）。发送时驱动 `copy_from_user` 一次就把数据写进这块双重映射区——**接收方读自己的映射就拿到了数据，无需第二次拷出**。对照图右下：`PROT_READ`（接收方不需要写）、`MAP_NORESERVE`（百万进程不为每份预留 swap）、`MAP_PRIVATE`（写时复制隔离）——三个参数各自是一道安全或内存题的答案。

1.3 血统：从 OpenBinder 到 binderfs

Binder 不是 Linux 亲生子：它的直系祖先是 Palm 的 OpenBinder（Dianne Hackborn 主导），2005 年随 Android 落户 Linux 内核（`drivers/android/binder.c`），此后主线化抗争十几年——至今仍以“Android 专属驱动”身份活在 staging 之外。2019 年的 binderfs（本树 UAPI `binderfs.h`）让容器/多用户环境能按需实例化 binder 设备节点，三域（binder/hwbinder/vndbinder）从此可以动态分配而非写死三个 misc 设备。读系统代码时懂一点血统史，能解释很多“为什么它长这样”的怪癖——Binder 的面向对象句柄模型、上下文管理者设计，都是 OpenBinder 时代就定下的基因。

1.4 安全模型对比：为什么“能传对象”如此重要

传统 IPC 的权限模型止步于“连上了没有”；Binder 的能力模型（capability）把**对象引用本身当作权限凭证**：拿到某服务的 IBinder，才有资格调用它；而 UID/PID 校验、SELinux 上下文随**每一次事务**重新采集。两相结合，Android 的安全叙事完整成立——“你能不能调”由名录与 SELinux 决定，“你是谁”由驱动决定，“你能干什么”由服务端代码决定。理解这三层职责切分，是审阅任何系统服务安全设计的起点。

源码锚点 · 第一章锚点

ProcessState.cpp:47 BINDER_VM_SIZE = 1MB - 2×PAGE_SIZE
ProcessState.cpp:48 DEFAULT_MAX_BINDER_THREADS = 15
ProcessState.cpp:503 mmap(nullptr, BINDER_VM_SIZE, PROT_READ, MAP_PRIVATE|MAP_NORESERVE, fd, 0)
external/kernel-headers/original/uapi/linux/android/binder.h UAPI 协议定义

第二章 总体架构：五层纵切

设计洞察 · 学习目标

1. 说出三个'每X一个'；2. 画出五层纵切图；3. 理解servicemanager角色



图 2 Binder 分层架构：客户端 Java/Native、内核驱动、服务端 Native/Java 的五层纵切与 ProcessState/IPCThreadState/servicemanager 三大件

三个“每 X 一个”记住主体结构：**每进程一个 ProcessState**（开驱动、mmap、线程池策略）、**每线程一个 IPCThreadState**（mIn/mOut 缓冲与命令收发）、**每服务端点一个 binder_node**（对端表现为 binder_ref 句柄）。servicemanager 是唯一的“名录”，句柄 0 固定指向它。

2.1 同步与 oneway 的一图两读

同一张五层图读两遍：同步调用的回程走⑨⑩（reply 原路返回）；oneway 在⑤把 `TF_ONE_WAY` 交给驱动后，⑥的入队走目标的 async_todo（对同目标保序、受 async 配额约束），发送方收到 `BR_TRANSACTION_COMPLETE` 即返回——**两条路的分歧点只有 flags 一个比特**。协议层的克制（不引入第二种事务结构）换来了实现层的统一：所有事务同构，仅靠队列与配额区分行为。

第三章 驱动协议：BC/BR 与核心结构

> **设计洞察 · 学习目标**
> 1. 列举BC/BR命令对；2. 解释五种数据结构关系；3. 描述transaction_stack压栈

3.1 命令对

Binder 协议是“双半工命令流”：应用写给驱动的 `BC_*` 与驱动推给应用的 `BR_*` 在同一个 `ioctl(BINDER_WRITE_READ)` 里双向捎带（`binder_write_read` 结构含 write_buffer/read_buffer 两半）。最重要的命令对：

BC（应用→驱动）	作用	对应 BR（驱动→应用）
BC_TRANSACTION	发起事务（同步或 oneway）	BR_TRANSACTION / BR_REPLY
BC_REPLY	服务端回包	BR_REPLY（唤醒原线程）
BC_INCREFS / BC_ACQUIRE / BC_RELEASE / BC_DECREFS	引用计数维护	BR_INCREFS / BR_ACQUIRE / BR_RELEASE / BR_DECREFS
BC_REQUEST_DEATH_NOTIFICATION	登记死亡通知	BR_DEAD_BINDER / BR_CLEAR_DEATH_NOTIFICATION_DONE
BC_ENTER_LOOPER / BC_REGISTER_LOOPER / BC_EXIT_LOOPER	线程池声明	BR_SPAWN_LOOPER（驱动要求扩池）
—（驱动主动）	—	BR_DEAD_REPLY / BR_FAILED_REPLY / BR_FROZEN_REPLY / BR_TRANSACTION_COMPLETE

特殊返回码就是排障词典：`BR_DEAD_REPLY`（对端进程死亡）、`BR_FAILED_REPLY`（事务被拒——常见于 async 空间耗尽）、`BR_FROZEN_REPLY`（目标进程被缓存冻结，`waitForResponse` :931 快速失败）。

3.2 数据结构关系

3.3 同步事务的驱动侧十步（文字展开）

结合 UAPI 与 libbinder 行为，`binder_transaction()` 处理一笔 BC_TRANSACTION 的路径可以展开为：

- **1 解析**：从 write_buffer 读出 `binder_transaction_data`（target.handle / code / flags / data.ptr / offsets.ptr）。
- **2 寻址**：handle=0 直取上下文管理者（servicemanager）；否则在当前 proc 的 refs 里查 desc → 目标 `binder_node`。
- **3 权限与上下文**：记录 `sender_pid / sender_euid`（凭 struct cred），附加 SELinux security_ctx 供 BR_TRANSACTION_SEC_CTX 使用。
- **4 分配**：在**目标进程**的 binder_alloc 里切一块 `binder_buffer`（同步走普通配额，oneway 先查 async_space）。
- **5 拷贝**：**copy_from_user(目标缓冲, data.ptr.buffer, size)**——全链路唯一一次拷贝；offsets 表里的每个 `flat_binder_object` 顺带翻译（本地 binder → 对端 handle，并为目标建 ref）。
- **6 线程选择**：目标 proc 找空闲 binder 线程；全忙且未超预算则唤醒待命线程并后续发 `BR_SPAWN_LOOPER`；无可用则入 proc todo 队列。
- **7 压栈**：同步事务把 `binder_transaction` 压入目标线程的 `transaction_stack`（reply 将沿栈返回原线程）。
- **8 唤醒**：目标线程 wait 队列唤醒；发送线程继续在 ioctl 里等读（写完命令未完事）。
- **9 回执**：发送线程读到 `BR_TRANSACTION_COMPLETE`；同步事务继续等 `BR_REPLY`，oneway 到此完结。
- **10 服务端收件**：目标线程读出 `BR_TRANSACTION(_SEC_CTX)` 与数据指针（就在它自己的 mmap 区），进入第五章 5.2 的分发。

把这十步与时序图（第九章）互相印证，驱动黑盒就变成了 checklist。

3.4 驱动核心数据结构关系图解（教科书画级）

text 五种数据结构的挂接关系（以 App→AMS 为例）

```
App 进程（客户端）  system_server 进程（服务端）
┌──────────────────┐ ┌──────────────────────┐
| binder_proc | | binder_proc |
| ├─threads(RB) | | ├─threads(RB) |
| | └─binder_thread| | | └─binder_thread |
| | └─transaction_stack | | └─transaction_stack|
| | └─binder_transaction| | └─binder_transaction|
| ├─refs_by_desc | | ├─nodes(RB) |
| | └─binder_ref | | └─binder_node |
| | desc=handle | | ptr/cookie →──┼──→ BBinder(JavaBBinder)
| | node ─────────┼──跨进程──→| refs ←─────────┼── binder_ref 挂这
```

```
| └─buffers | | └─buffers |
| (mmap 1MB-2页)| | (mmap 1MB-2页) |
└──────────────┘ └──────────────┘

// 讲解注释①：App 侧拿到的是 binder_ref（desc=32位句柄）
// Java 层包装为 BpBinder → BinderProxy
// 讲解注释②：AMS 侧注册的是 binder_node（ptr/cookie指向 JavaBBinder）
// Java 层就是 Binder.onTransact 的接收者
// 讲解注释③：驱动负责两端的 ref↔node 映射——
// App 的 handle 在驱动里找到 AMS 的 node → 路由到正确进程
```

设计洞察 · 逐段精读：为什么这五种结构是"最小完备集"

proc（进程边界）、thread（执行流+调用栈）、node（服务端点）、ref（客户端引用）、buffer（数据载体）——**每种对应一个跨进程通信的必要维度**。少任何一种，通信就不完整：没有 thread 就无法并发、没有 ref 就无法寻址、没有 buffer 就没有数据。这是"数据结构=问题域的最小模型"的教科书案例。

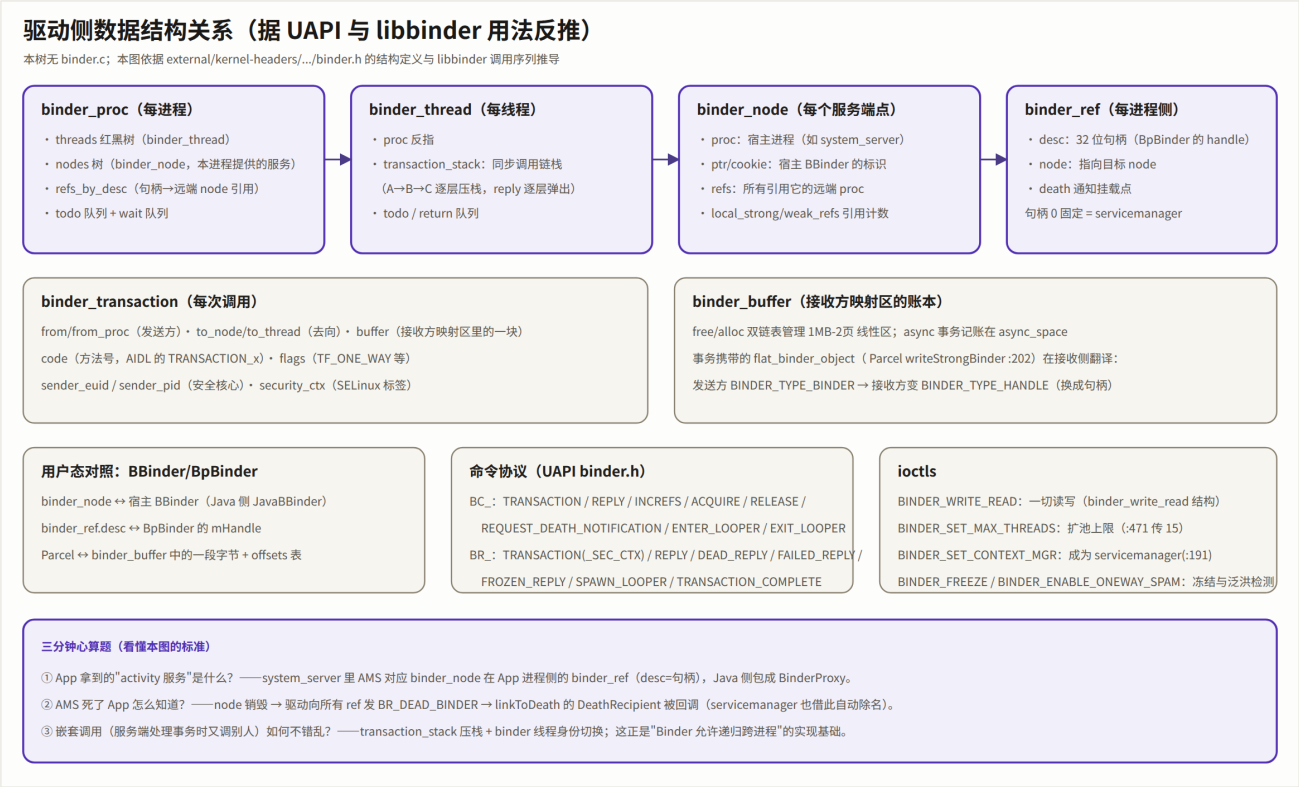


图 3 驱动侧数据结构关系：proc/thread/node/ref/buffer/transaction 的挂接方式与用户态 BBinder/BpBinder 对照（UAPI 反推）

三个最值得内化的关系：**句柄的本质**是接收进程侧 `binder_ref.desc`，Java 世界的 `BinderProxy` 不过是它的马甲；**同步调用链**靠每个线程的 `transaction_stack` 压栈维持，因此 binder 允许跨进程递归（A→B→C 再逐层返回）；**死亡通知**挂在 ref 上，node 销毁时驱动向所有引用方广播 `BR_DEAD_BINDER`——第七章你会看到 servicemanager 如何用这一机制自动除名。

第四章 ProcessState：进程的 Binder 资产

cpp frameworks/native/libs/binder/ProcessState.cpp:485-517（节选）

```
ProcessState::ProcessState(const char* driver)
: mDriverFD(-1), mVMStart(MAP_FAILED),
mMaxThreads(DEFAULT_MAX_BINDER_THREADS), ... {
base::Result<int> opened = open_driver(driver); // open("/dev/binder")
if (opened.ok()) {
// 把一块虚拟地址空间交给内核做事务接收区
mVMStart = mmap(nullptr, BINDER_VM_SIZE, PROT_READ,
MAP_PRIVATE | MAP_NORESERVE, opened.value(), 0);
...
}
LOG_ALWAYS_FATAL_IF(!opened.ok(), "Binder driver '%s' could not be opened..."); // Android 上
bi
nder 不可缺
}
```

open_driver（:452）里还有一处必读：ioctl(fd, BINDER_SET_MAX_THREADS, &maxThreads)（:470-471）把 15 的上限告诉驱动——**扩池的决策方是驱动**（按并发压力发 BR_SPAWN_LOOPER），用户态只声明预算。getContextObject（:144）返回句柄 0（servicemanager）的 BpBinder，这是“找服务”的起点。

第五章 IPCThreadState：命令收发中枢

设计洞察 · 学习目标

1. 画出transact/waitForResponse流程；2. 理解身份切换与恢复；3. 追踪BR_TRANSACTION分发

5.1 发起端：transact 与 waitForResponse

cpp frameworks/native/libs/binder/IPCThreadState.cpp:706-776（节选）

```
status_t IPCThreadState::transact(int32_t handle, uint32_t code,
const Parcel& data, Parcel* reply, uint32_t flags) {
// 讲解注释①：TF_ACCEPT_FDS——本事务允许对端"顺手"传文件描述符过来
// （位图/FDA 的接收许可，写方无需声明、读方默认接受）。
flags |= TF_ACCEPT_FDS;
err = writeTransactionData(BC_TRANSACTION, flags, handle, code, data, nullptr); // :725 组包
if ((flags & TF_ONE_WAY) == 0) {
if (mCallRestriction != ProcessState::CallRestriction::NONE) { ... } // :733 oneway 管制检查
if (reply) {
err = waitForResponse(reply); // :751 同步：等 BR_REPLY
} else {
Parcel fakeReply;
err = waitForResponse(&fakeReply);
}
} else {
```

```
err = waitForResponse(nullptr, nullptr); // :772 oneway：收到 BR_TRANSACTION_COMPLETE 即止
}
```

mCallRestriction 是个漂亮的细节：进程可以把自己配置成“禁止同步外呼”（FATAL_IF_NOT_ONEWAY）——servicemanager 正是如此（第七章），从机制上杜绝名录服务被慢客户端的同步调用卡死。

waitForResponse（:897-990）是个命令泵：循环 talkWithDriver 读 BR_*，直到拿到本事务的结果。BR_REPLY 分支（:939-960）把驱动移交的缓冲区用 ipcSetDataReference **零拷贝**包装进 reply Parcel——freeBuffer 回调负责用完归还驱动。oneway 的终点在 BR_TRANSACTION_COMPLETE（:921）：它只表示“内核已收件”，不表示对端已执行。

5.2 服务端：executeCommand 与身份切换

cpp frameworks/native/libs/binder/IPCThreadState.cpp:1227-1273（节选）

```
case BR_TRANSACTION_SEC_CTX:
case BR_TRANSACTION: {
binder_transaction_data_secctx tr_secctx;
... // 读事务数据（缓冲区就在本进程映射区）
Parcel buffer;
buffer.ipcSetDataReference(tr.data.ptr.buffer, ...); // :1240 零拷贝包装
const pid_t origPid = mCallingPid; // :1248 保存旧身份
const uid_t origUid = mCallingUid;
...
mCallingPid = tr.sender_pid; // :1269 内核采集的调用方身份
mCallingUid = tr.sender_euid; // :1270 —— 一切权限检查的锚点
mCallingSid = reinterpret_cast<const char*>(tr_secctx.secctx); // :1271 SELinux 上下文
```

恢复与嵌套：方法执行完按 orig 值恢复身份；若服务端在处理事务时又调用第三方，transaction_stack 压栈保证返回链正确——身份与调用链都在线程上，这就是 Binder“跨进程递归”的秘密。

5.3 talkWithDriver：一次 ioctl 的行李清单

cpp frameworks/native/libs/binder/IPCThreadState.cpp:997-1100（节选）

```
status_t IPCThreadState::talkWithDriver(bool doReceive) {
binder_write_read bwr;
const bool needRead = mIn.dataCapacity() == 0 ...;
bwr.write_size = outAvail; // mOut 里积攒的 BC_* 命令
bwr.write_buffer = (uintptr_t)mOut.data();
if (needRead && doReceive) {
bwr.read_size = mIn.dataCapacity();
bwr.read_buffer = (uintptr_t)mIn.data(); // 空出的缓冲供驱动回填 BR_*
} else { bwr.read_size = 0; ... }
...
do { ioctl(mProcess->mDriverFD, BINDER_WRITE_READ, &bwr); } while (...);
```

mIn/mOut 是线程的两只手：出手（BC 命令）与接手（BR 命令）在同一 ioctl 里双向结清。**命令是批量捎带的**——几次 transat 的引用计数命令可能搭同一班车。这也是读 binder 日志的要点：一次 ioctl ≠ 一次事务。

5.4 引用计数命令：INKREFS 家族的日常

executeCommand 不只处理事务：BR_ACQUIRE/BR_INCREFS/BR_RELEASE/BR_DECREFS（对应客户端发出的 BC_* 计数命令）驱动着对象生命周期——sp<IBinder> 持有即 BC_ACQUIRE，析构即 BC_RELEASE；weak 计数走 INCREFS/DECREFS 对。当你在日志里看到成串的 IncRefs/DecRefs，那是一个 BpBinder 正在被缓存收放。**Binder 的“自动内存管理”本质是双端计数 + 驱动仲裁，**Java 层的 BinderProxy 只是把这套计数翻译成了 GC 世界的 WeakReference + 死亡通知。

3.5 BINDER_WRITE_READ ioctl 深度走读

Binder 的一切通信都经过一个 ioctl 调用——这是**最核心的系统调用**。

ioctl 的行李清单

cpp frameworks/native/libs/binder/IPCThreadState.cpp:997 起（骨架+逐行注释）

```
status_t IPCThreadState::talkWithDriver(bool doReceive) {
// 讲解注释①：binder_write_read 是 ioctl 的唯一载荷结构
struct binder_write_read bwr;
// 讲解注释②：写半区（应用→驱动）
bwr.write_size = outAvail; // mOut 中积攒的命令字节数
bwr.write_buffer = (uintptr_t)mOut.data(); // 指向 BC_* 命令流
// 典型内容：BC_TRANSACTION + binder_transaction_data（参数）
// 可能还搭 BC_INCREFS/BC_ACQUIRE（引用计数）
// 讲解注释③：读半区（驱动→应用）
if (needRead && doReceive) {
bwr.read_size = mIn.dataCapacity(); // 空出的缓冲供驱动回填
bwr.read_buffer = (uintptr_t)mIn.data();
// 驱动可能回填 BR_TRANSACTION（收到调用）
// 或 BR_REPLY（同步调用的返回）
// 或 BR_SPAWN_LOOPER（要求扩线程池）
// 或 BR_DEAD_BINDER（服务死了）
} else {
bwr.read_size = 0; // 只写不读（oneway 发送）
}
// 讲解注释④：一次 ioctl 完成双向通信
do {
ioctl(mProcess->mDriverFD, BINDER_WRITE_READ, &bwr);
} while (bwr.read_consumed > 0 && bwr.write_consumed < bwr.write_size);
// 讲解注释⑤：返回后处理
// mIn 中现在是 BR_* 命令流 → executeCommand() 分发
// mOut 中的 BC_* 已被驱动消费 → 清空
}
```

一次同步调用的 ioctl 序列

text App 调用 AMS.activityPaused() 的完整 ioctl 序列

```
App 进程（调用方）：
ioctl #1: write={BC_TRANSACTION} read={} → 发出请求，阻塞等回复
[驱动路由：写入 system_server 的 todo 队列，唤醒其 binder 线程]
[App 线程在 ioctl 中等待（内核挂起）]
system_server（服务方）：
```

```
ioctl #2: write={} read={BR_TRANSACTION} → 收到请求（之前阻塞在 ioctl 等读）
[执行业务代码]
ioctl #3: write={BC_REPLY} read={} → 发出回复
App 进程（唤醒）：
ioctl #1 返回：read={BR_REPLY} → 拿到结果
// 总结：一次同步调用 = App 1 次 ioctl（含等待）+ Server 2 次 ioctl
// 总计 3 次 ioctl（1 次阻塞等待 + 2 次数据传输）
```

设计洞察 · 逐段精读：为什么用 ioctl 而不是 read/write

① ioctl 是**全双工**（一个调用同时传写缓冲和读缓冲）——read/write 只能单向。② ioctl 可以传自定义结构体（binder_write_read），read/write 只能传字节流。③ ioctl 进入内核后可以直接操作驱动的私有数据结构（binder_proc），不需要额外的系统调用。**"用一个 ioctl 完成一次全双工事务"是 Binder 高效的核心之一**。

源码锚点 · 第五章锚点

IPCThreadState.cpp:706 transact / :725 writeTransactionData(BC_TRANSACTION)
:897 waitForResponse / :921 BR_TRANSACTION_COMPLETE / :939 BR_REPLY
:997 talkWithDriver（ioctl BINDER_WRITE_READ）
:1147 executeCommand / :1227 BR_TRANSACTION_SEC_CTX / :1269-1273 身份装载

第六章 Parcel：跨进程的类型系统

设计洞察 · 学习目标

1. 解释flat_binder_object二态；2. 说出事务即域；3. 列举常用读写API

Parcel 是双方约定的字节格式（含接口 token 校验、数据区与 offsets 表），`Parcel.cpp` 定义了全部读写。三个重点：

- **writeStrongBinder**（`:202` 构造 `flat_binder_object`）：把一个 IBinder “降维”成 {type, binder/cookie 或 handle} 结构。发送方的 `BINDER_TYPE_BINDER` 在驱动里被翻译成接收方的 `BINDER_TYPE_HANDLE`——**跨进程传递的是句柄不是指针**，收到方 `readStrongBinder` 后拿到的其实是 BpBinder 代理。
- **fd 传递**（`BINDER_TYPE_FDA`）：驱动为目标进程复制文件描述符表项——位图、ANR trace 管道都靠它跨进程。
- **writeInterfaceToken**：每笔事务头部写入接口名与严格模式策略，服务端 `enforceInterface` 校验——**协议版本对齐**的最小机制。

6.2 writeStrongBinder：对象降维现场

cpp frameworks/native/libs/binder/Parcel.cpp:195-215（节选）

```
status_t Parcel::writeStrongBinder(const sp<IBinder>& val) {
...
flat_binder_object obj;
obj.flags = acceptFds ? FLAT_BINDER_FLAG_ACCEPTS_FDS : 0;
if (val == nullptr) { obj.type = BINDER_TYPE_BINDER; obj.binder = 0; obj.cookie = 0; }
else if (val->localBinder()) {
obj.type = BINDER_TYPE_BINDER; // 本地对象：传指针+cookie
obj.binder = reinterpret_cast<uintptr_t>(val->getRefAssoc());
obj.cookie = reinterpret_cast<uintptr_t>(val.get());
} else {
obj.type = BINDER_TYPE_HANDLE; // 远端代理：传句柄
obj.handle = val->remoteBinder()->handle();
}
return finishFlattenBinder(val, obj); // 记入 offsets 表
}
```

同一个方法两种编码：**本地对象带指针、远端引用带句柄**，驱动负责翻译。offsets 表让接收方能跳着扫出所有对象槽位——Parcel 的“数据平面”与“对象平面”由此分离。常用读写速查：`writeInterfaceToken/enforceInterface`（协议对齐）、`writeInt32/Int64/Float`、`writeString16`、`writeStrongBinder`、`writeFileDescriptor`（FDM 数组）、`writeBlob`（可 zero-copy 的字节块）。**读序必须与写序严格对称**——aidl 生成代码存在的意义就是替你保证这一点。

6.1 一个易被忽视的真相：Parcel 不是序列化框架

Parcel 有生命周期约定：服务端在事务里收到的 data/reply 由 binder 缓冲区背书，**事务返回即失效**。跨事务保存 Parcel 或在异步路径上复用它是系统代码经典 bug 源。记住口诀：**事务即域（transaction-scoped）**。

第七章 servicemanager：名录与门卫

设计洞察 · 学习目标

1. 解释零线程池设计；2. 列出注册五道关卡；3. 追踪句柄0自举



图 4 servicemanager 的自举、注册五道关卡、查询与三域隔离，附“为什么这么瘦”四则

cpp frameworks/native/cmds/servicemanager/main.cpp:113-143（节选）

```cpp
int main(int argc, char** argv) {
const char* driver = argc == 2 ? argv[1] : "/dev/binder";
sp<ProcessState> ps = ProcessState::initWithDriver(driver);
ps->setThreadPoolMaxThreadCount(0); // :125 零工作线程
ps->setCallRestriction(ProcessState::CallRestriction::FATAL_IF_NOT_ONEWAY); // :126
sp<ServiceManager> manager = sp<ServiceManager>::make(std::make_unique<Access>());
if (!manager->addService("manager", manager, ...).isOk()) { ... } // :129 自举注册
IPCThreadState::self()->setTheContextObject(manager);
ps->becomeContextManager(); // :134 ioctl(BINDER_SET_CONTEXT_MGR)
sp<Looper> looper = Looper::prepare(false);
BinderCallback::setupTo(looper); // binder fd 挂进 Looper
while (true) looper->pollAll(-1); // :141-143 单线程永续
}
```

`addService`（`ServiceManager.cpp:295-330`）的五道关卡值得背下来：app uid 拒绝（`:298`）→ SELinux `canAdd`（`:302`）→ 名字合法性 → VINTF 声明一致性（`:317`，HAL 服务必须与设备清单吻合）→ `linkToDeath` 挂死亡通知（`:325`，**服务死则自动除名**）。`getService`（`:232`）同样过 SELinux `canFind`——名录不只是字典，还是访问控制的第一道门。

第八章 Java 层与 AIDL：对象的两个世界

7.1 客户端侧：ServiceManager.java 的缓存链

Java 侧查询走 `ServiceManager.getService("activity")`：先查进程内 `sCache`（ByName map）→ 未命中经 `ServiceManagerNative.asInterface(BinderProxy(0))` 向名录发 **oneway** getBinder 请求（`ServiceManagerRaw.java` 的 getAsync 模式）→ 结果写缓存并回调。**句柄 0 是唯一天生可用的代理**——名录自身的地址被硬编码进协议，其他一切服务都从它出发。缓存的意义：名录查询从“每服务一次”降为“每进程每服务一次”；代价是服务死亡后要靠死亡通知主动失效（第九章 runtime restart 场景）。

8.1 Binder / BinderProxy 与 JNI 桥

Java 世界只有两个类需要真正理解：**Binder（本地服务端基类）** 与 **BinderProxy（远端代理）**。它们与 native 世界的缝合线在 `android_util_Binder.cpp`：

cpp frameworks/base/core/jni/android_util_Binder.cpp:749-790（节选）

```
jobject javaObjectForIBinder(JNIEnv* env, const sp<IBinder>& val) {
// 讲解注释①：checkSubclass 是"本地对象吗"的试金石——
// 本进程 JavaBBinder 携带 gBinderOffsets 印记，命中则无需代理。
if (val->checkSubclass(&gBinderOffsets)) {
// 讲解注释②：绕过一切包装直接还给调用者原 Java 对象——
// "自己进程里的服务调用自己"零开销的秘密（asInterface 本地直连）。
return static_cast<JavaBBinder*>(val.get())->object(); // :758
}
BinderProxyNativeData* nativeData = new BinderProxyNativeData();
nativeData->mOrgue = new DeathRecipientList;
nativeData->mObject = val; // :763-765 持有 BpBinder
jobject object = env->CallStaticObjectMethod(gBinderProxyOffsets.mClass,
gBinderProxyOffsets.mGetInstance, (jlong) nativeData, ...); // :767 创建/复用 BinderProxy
...
uint32_t numProxies = gNumProxies.fetch_add(1, ...); // :776 进程内代理计数
if (numProxies >= numLastWarned + PROXY_WARN_INTERVAL) {
ALOGW("Unexpectedly many live BinderProxies: %d", numProxies); // 泄漏可观测
```

三个设计点：**同 IBinder 每进程一个 BinderProxy**（getInstance 按 nativeData 复用，防止代理膨胀）；**BinderProxy 持 native 指针、native 侧持弱引用回 Java**（`JavaBBinderHolder` :462，双向生命周期由 DeathRecipientList 缝合）；**代理计数告警**——BinderProxy 泄漏（忘记 unlinkToDeath、缓存未清理）从“静默吃内存”变成日志可查。

方向反过来（`ibinderForJavaObject` :793）：Java Binder → `JavaBBinderHolder` 弱缓存的 JavaBBinder；Java BinderProxy → 直接取回 native BpBinder。`BinderProxy.transact`（`BinderProxy.java:526` → native `:1387`）就是这两张查找表的日常运转。

8.2 AIDL 生成物解剖

以框架真实的 `IActivityManager.aidl`（`core/java/android/app/`，接口声明 `:89`）为例，aidl 编译器生成三件套：

- **Stub**（服务端）：继承 Binder，`onTransact(code, data, reply, flags)` 里按 code 分发——每个方法一个 `TRANSACTION_x` 常量；先 `data.enforceInterface(DESCRIPTOR)` 校验协议，再逐参数 `data.readXXX()`，最后 `reply.writeXXX()`。
- **Stub.Proxy**（客户端）：持有 `mRemote`（IBinder，实为 BinderProxy），每个方法：`_data.writeInterfaceToken` + 参数 → `mRemote.transact(TRANSACTION_x, _data, _reply, 0)` → 读返回值。`asInterface(IBinder)` 按是否本进程对象决定返回本地实现还是 Proxy。
- **Parcelable 参数**：aidl 定义的 `in/out/inout` 决定数据流向——`out` 参数在服务端 new 空对象、由 reply 回填。

手写一个 MiniBinder 服务（不复用 SystemService）只需要：定义 aidl → 服务端 **new Stub() { 实现 }** 并 `ServiceManager.addService("mysvc", this)` → 客户端 `IMySvc.Stub.asInterface(ServiceManager.getService("mysvc"))`。三行代码背后，正是本册第二到第八章的全部机器。

8.3 Stub/Proxy 生成物还原（典型结构）

aidl 编译产物可用约三十行还原其骨架（依 aidl 语义手写，标注与源码对应关系）：

java IActivityManager 生成物的典型结构（依 aidl 语义还原）

```
public interface IActivityManager extends android.os.IInterface {
public static abstract class Stub extends android.os.Binder
implements IActivityManager {
static final int TRANSACTION_activityPaused = FIRST_CALL_TRANSACTION + n;
public boolean onTransact(int code, Parcel data, Parcel reply, int flags) {
data.enforceInterface(descriptor); // 协议对齐
switch (code) {
case TRANSACTION_activityPaused: {
IBinder b = data.readStrongBinder(); // 读参数（按序！）
long token = data.readLong();
activityPaused(b, token); // 调真身
reply.writeNoException(); // 回执
return true;
} ...
}
}
private static class Proxy implements IActivityManager {
private IBinder mRemote; // 实为 BinderProxy
public void activityPaused(IBinder token, long pausedAt) {
Parcel _data = Parcel.obtain(), _reply = Parcel.obtain();
_data.writeInterfaceToken(descriptor);
_data.writeStrongBinder(token); _data.writeLong(pausedAt);
mRemote.transact(TRANSACTION_activityPaused, _data, _reply, 0);
_reply.readException(); // 同步等结果
}
}
public static IActivityManager asInterface(IBinder obj) {
if (obj instanceof IActivityManager) return (IActivityManager) obj; // 本地直连
return new Proxy(obj); // 远端包代理
}
}
```

```
}
```

读真实生成代码（`out/soong/.intermediates/.../IActivityManager.java`，本树 out/ 若有构建产物可直接翻）会发现更多工程细节：`_data.recycle()` 归还 Parcel 池、异常经 `writeException` 编码回传、oneway 方法没有 `_reply`。**手写一遍这个骨架**是检验你是否真懂本册前八章的最佳练习。

9.3 Binder 性能调优实战（教科书级教程）

何时需要调优

text Binder 性能问题的典型信号

```
① dumpsys binder_calls_stats 某接口 >1ms 均值
② perfetto binder_track 轨有持续长条
③ ANR traces 显示"waiting to send binder"或"binder thread full"
④ logcat 频繁 "binder: ... transaction failed"
```

五步调优法

bash

```
# 步骤 1：定位热点接口
adb shell dumpsys binder_calls_stats --enable
# ... 运行场景 ...
adb shell dumpsys binder_calls_stats | grep -E "mean_ms|count"
# 找到 mean_ms 最高的接口
# 步骤 2：抓取事务级 trace
# perfetto 配置开启 binder_track
adb shell perfetto -c /data/misc/perfetto-configs/binder_trace.txt -o
/data/misc/perfetto-traces/tra
ce
# 步骤 3：分析事务链
# 在 trace 中找到目标接口 → 看唤醒-执行-回复三段耗时
# 唤醒慢 = 目标进程 busy（不是 Binder 本身慢）
# 执行慢 = 业务代码慢
# 回复慢 = 数据序列化大
# 步骤 4：定向优化
# 场景 A：调用太频繁 → 批量化接口（一次传一批 vs N 次单传）
# 场景 B：数据太大 → 改用 fd + SharedMemory（如位图）
# 场景 C：目标线程池满 → 增大 maxThreads 或减少同步调用
# 步骤 5：复验
# 重新跑 dumpsys binder_calls_stats 对比优化前后
```

常见优化场景

场景	原因	方案	效果
App 高频查询 AMS	每帧一次 getService	SystemServiceRegistry 缓存	消除 95% 调用
大位图跨进程	Parcel 序列化慢	fd + SharedMemory	10x 加速
system_server 线程满	31 个线程全忙	减少嵌套同步调用	消除阻塞

场景	原因	方案	效果
oneway 塞满 async space	高频 oneway 大数据	分片或降频	避免 FAILED_REPLY

> **设计洞察 · 逐段精读：Binder 慢 ≠ Binder 的错**
>
> 十跳中真正可控的是"业务执行"（第八跳）和"数据序列化"（第二跳和第九跳）。**Binder 协议本身（驱动+IPCThreadState）极少是瓶颈**——它只占 10-50μs。大部分"Binder 慢"其实是服务端业务慢或客户端序列化慢。先用 perfetto 定位是哪一段慢，再定向优化。

8.4 AIDL 生成物完整解剖（教科书级逐步分析）

AIDL 编译器从 .aidl 文件自动生成 Java/C++ 绑定代码。以下以 `IActivityManager.aidl` 的一个方法为例，完整解剖生成物。

输入：.aidl 文件

java frameworks/base/core/java/android/app/IActivityManager.aidl（节选）

```
interface IActivityManager {
// 讲解注释①：接口声明
void activityPaused(in IBinder token, long pausedAt);
// 参数修饰符：in=只传过去 / out=只传回来 / inout=双向
}
```

输出1：Stub（服务端存根）

java AIDL 编译器生成的 Stub（骨架+逐行注释）

```
public static abstract class Stub extends android.os.Binder
implements IActivityManager {
// 讲解注释②：DESCRIPTOR 是接口的唯一标识
private static final String DESCRIPTOR = "android.app.IActivityManager";
// 讲解注释③：每个方法分配一个事务码（从 FIRST_CALL_TRANSACTION 递增）
static final int TRANSACTION_activityPaused = (android.os.IBinder.FIRST_CALL_TRANSACTION +
0);
// 讲解注释④：asInterface——本地直连或远程代理的分水岭
public static IActivityManager asInterface(android.os.IBinder obj) {
if (obj == null) return null;
IActivityManager in = (IActivityManager)obj.queryLocalInterface(DESCRIPTOR);
if (in != null) return in; // 本进程 → 直接返回本地实现
return new Stub.Proxy(obj); // 远程 → 包装代理
}
// 讲解注释⑤：onTransact——服务端的"路由器"
@Override
public boolean onTransact(int code, Parcel data, Parcel reply, int flags) {
switch (code) {
case TRANSACTION_activityPaused: {
// 讲解注释⑥：协议校验（防止版本不匹配）
data.enforceInterface(DESCRIPTOR);
// 讲解注释⑦：按序读取参数（与 Proxy 的写入序严格对称）
android.os.IBinder _arg0 = data.readStrongBinder();
long _arg1 = data.readLong();
```

```
// 讲解注释⑧：调用真实实现（AMS 的方法）
this.activityPaused(_arg0, _arg1);
// 讲解注释⑨：写回执
reply.writeNoException();
return true;
}
}
return super.onTransact(code, data, reply, flags);
}
}
```

输出2：Stub.Proxy（客户端代理）

java AIDL 编译器生成的 Proxy（骨架+逐行注释）

```
private static class Proxy implements IActivityManager {
private android.os.IBinder mRemote; // 实际是 BinderProxy
@Override
public void activityPaused(android.os.IBinder token, long pausedAt)
throws android.os.RemoteException {
// 讲解注释⑩：准备两个 Parcel
android.os.Parcel _data = android.os.Parcel.obtain();
android.os.Parcel _reply = android.os.Parcel.obtain();
try {
// 讲解注释⑪：写入协议标识+参数（与 Stub 的读取序严格对称）
_data.writeInterfaceToken(DESCRIPTOR);
_data.writeStrongBinder(token);
_data.writeLong(pausedAt);
// 讲解注释⑫：发起 Binder 调用（书3 十跳的第一跳）
mRemote.transact(Stub.TRANSACTION_activityPaused, _data, _reply, 0);
// 讲解注释⑬：读取异常+返回值
_reply.readException();
} finally {
// 讲解注释⑭：归还 Parcel（池化复用，不产生 GC）
_reply.recycle();
_data.recycle();
}
}
}
```

生成物中隐藏的 10 个设计决策

设计	生成代码中的体现	为什么
DESCRIPTOR 字符串	writeInterfaceToken/enforceInterface	版本兼容：新旧版本接口共存
FIRST_CALL_TRANSACTION 递增码	TRANSACTION_activityPaused = 0	一个 Binder 对象可以承载多个接口
asInterface 本地判断	queryLocalInterface	同进程直连省一次 Binder 开销
Parcel 池化	obtain/recycle	避免高频 GC

设计	生成代码中的体现	为什么
in/out/inout	生成代码中读写顺序不同	数据方向=写入侧写、读取侧读
异常编码	writeException/readException	跨进程异常不能直接 throw
线程安全	Proxy 无锁	每个线程独立 IPCThreadState
oneway 编译	flags 参数=ThreadFlags.FLAG_ONEWAY	编译期确定是否阻塞
默认实现	default 方法	新方法不破坏老客户端
raw/bool 转 int	writeInt/readInt	Parcel 只支持基础类型

设计洞察 · 逐段精读：为什么要自动生成

同一套模式（token→参数→transact→异常→回收）重复 140+ 次（IActivityManager 的方法数）——**手写必出错、自动生成保证一致性**。这就是"代码生成器"的核心价值：把可模式化的东西交给机器。理解了 AIDL 生成物的结构，你就掌握了 Android 接口定义的全部秘密。

源码锚点 · 第八章锚点

Binder.java:330/340 getCallingPid/getCallingUid（native 直读 IPCThreadState）
Binder.java:397 clearCallingIdentity / restoreCallingIdentity
BinderProxy.java:526/584/614 transact → transactNative
android_util_Binder.cpp:749 javaObjectForIBinder / :793 ibinderForJavaObject / :1387 transact JNI
IActivityManager.aidl:89 interface 声明（39KB，140+ 方法——AMS 面向全机的合同书）

第九章 端到端全链路：把图走成路

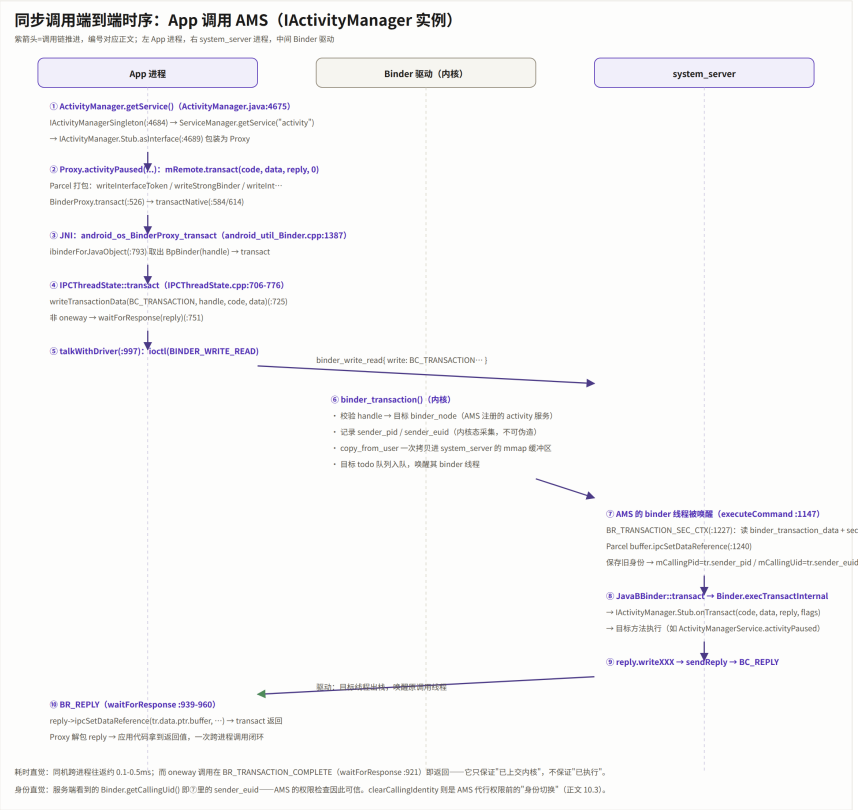


图 5 同步调用端到端时序：App 调用 AMS 的十跳全景（含驱动的采集与拷贝、服务端身份装载）

配合时序图逐跳复述（正文不再重复细节，只补三处“图上放不下”的注释）：

- **第①跳**的 `IActivityManagerSingleton`（`ActivityManager.java:4684-4690`）是进程级单例缓存——`getService` 只在首次发生，此后直连句柄。**服务重启后句柄失效**靠 BinderProxy 死亡通知触发缓存刷新，这也是“system_server 重启（runtime restart）后 App 要重新拿服务”的机制层解释。
- **第⑥跳**驱动里除了拷贝与排队，还要做**目标线程选择**：优先找空闲 binder 线程，全忙则看是否超过线程预算决定扩池（`BR_SPAWN_LOOPER`）或排队。因此“服务端线程打满”不是异常，是供给不足的常态信号。
- **第⑨跳**的 BC_REPLY 走的是与第⑥跳对称的路径，但**不新建缓冲**：回复复用调用方事务的预留空间，往返的内存账本始终只有一本。

第十章 线程池与高级机制

> **设计洞察 · 学习目标**
>
> 1. 画出16线程模型；2. 解释oneway三误区；3. 追踪死亡通知链



图 6 Binder 线程池：来源（主线程+驱动扩池）、一生（双面性）、三条铁律与冻结/泛洪新语义

10.1 死亡通知：优雅地处理“对方没了”

`linkToDeath(DeathRecipient)` 走 `BC_REQUEST_DEATH_NOTIFICATION`，目标 node 销毁时驱动回推 `BR_DEAD_BINDER`。框架里最著名的用户是 AMS：每个应用进程的 `ProcessRecord` 都挂 `AppDeathRecipient`（`ProcessList.java:5056-5058` 创建、`:1648/4937` 解绑）——**App 被杀后 AMS 收到回调清理组件与窗口**，用户感知的“应用重启”从这里开始。servicemanager 的自动除名（第七章）是同一机制的另一个客户。

10.2 身份切换：clearCallingIdentity 的用途

服务端方法里 `Binder.getCallingUid()` 是调用方身份；但当 AMS **代表系统**去做下一步动作（如启动 Activity）时，必须先 `clearCallingIdentity()`（`Binder.java:397`）切回本进程身份，事毕 `restoreCallingIdentity` 恢复。忘切 = 把调用方权限放大给系统（提权漏洞）；忘恢复 = 后续调用者身份错乱。**所有做系统服务的同学都要把这组 API 写进肌肉记忆**。

10.2.1 身份切换代码示例

java 服务端典型模式（AMS 各方法俯拾皆是）

```
public void startInstrumentation(...) {
final int callingUid = Binder.getCallingUid(); // 调用方身份（驱动背书）
enforceCallingPermission(...) // 用调用方身份做检查
final long ident = Binder.clearCallingIdentity(); // 切换为 system_server 本身
try {
mContext.startActivityAsUser(..., Process.myUserHandle()); // 代行系统权限
} finally {
```

```
Binder.restoreCallingIdentity(ident); // 必须恢复：后续事务身份不可污染
}
}
```

规则化表述：**检查用调用方身份，动作用系统身份，边界必须 try/finally**。这两组 API 的不对称性（clear 返回 token、restore 消费 token）就是为强制成对使用而设计。

10.3 oneway 的真实语义

三个常被误解的点：oneway **不保证对端已执行**（只到 BR_TRANSACTION_COMPLETE）；对**同一目标进程**的 oneway 在驱动内保序（单 async 队列），跨进程自然可能乱序；async 空间（映射区约一半）是独立预算，塞满则 BR_FAILED_REPLY——大位图走 oneway 会翻车的根源。Android 13 还加了泛洪检测 BR_ONEWAY_SPAM_SUSPECT（waitForResponse:915）：异常高频的 oneway 发送者会被点名进日志。

10.4 冻结语义（Android 12+）

缓存应用被 cgroup freezer 冻结时，发往它的**同步**事务立刻 BR_FROZEN_REPLY（:931）而非挂起等超时——把“冻结进程的 binder 行为”从模糊变成确定，是后台冻结功能能落地的协议前提。

10.5 libbinder_ndk 与稳定性等级

现代 C++ 服务（包括 AIDL HAL）多走 **libbinder_ndk**（frameworks/native/libs/binder/ndk）：AIBinder/AStatus/AIBinder_setInheritRtl? 这套 C API 把 libbinder 包成可由 NDK 稳定暴露的薄层，aidl 编译器可为同一接口产出 backend = java/cpp/ndk/rust 四份绑定——**接口一次定义，四种语言的客户端与服务端互通**。配套的**稳定性等级**（Stability / VINTF）：none（进程内）→ vendor（vendor 分区接口需 VINTF 登记版本）→ system（可跨分区）→ vintf（最高承诺）。等级决定接口能否演化、跨哪些分区可用——给系统加服务时，选错等级比写错代码更难补救。

10.6 线程栈实勘

text debuggerd -b <system_server_pid>（binder 线程栈节选，示意）

```
"Binder:1234_5" ... sys_waitid / ioctl(/dev/binder) ← 空闲：阻塞在 talkWithDriver
"Binder:1234_9" ... IPCThreadState::transact ← 正外呼他人（出站）
→ android_os_BinderProxy_transact
→ com.android.server.am.XXX.method ← Java 业务栈
"Binder:1234_12" ... BBinder::transat ← 入站：正处理别人的事务
→ IActivityManager.Stub.onTransact
```

空闲线程沉在 ioctl、出站线程停在 transact、入站线程挂在业务——三类栈型一眼可辨。ANR/卡顿分析时先数 binder 线程构成，再定位慢在哪一型，是系统性能的基本功。

第十一章 设计决策十二问

- **为什么 mmap 接收区而不是共享内存池？** 每进程私有 1MB 线性区，配额隔离、免全局锁；PROT_READ 只读防篡改（第一章）。
- **为什么 15+1 线程上限？** 防 binder 线程吃光进程内存；供给制（驱动 SPAWN_LOOPER）按需扩容（第十章）。
- **为什么身份在驱动采集？** 用户态任何“自报身份”都可伪造；`sender_euid` 是权限体系的信任根（第五章）。
- **为什么 oneway 有独立空间？** 无限 oneway 会淹没接收方；async 半区配额把洪泛伤害封顶（第三章/十章）。
- **为什么每进程一个 BinderProxy？** 防代理膨胀 + 泄漏可观测（第八章）。
- **为什么 servicemanager 零线程池、仅 oneway？** 名录必须极简极稳，绝不能被客户端行为牵连（第七章）。
- **为什么句柄 0 自举？** “找服务”本身也是服务——驱动把名录固定在特殊句柄，打破循环依赖（第七章）。
- **为什么注册即 linkToDeath？** 名录与进程生命周期同步，杜绝“名录里的僵尸服务”（第七章）。
- **为什么同步用线程栈而非全局表？** 栈天然支持跨进程递归调用链，无需回滚逻辑（第三章）。
- **为什么 AIDL 而非手写 Parcel？** 协议生成保证两端一致（enforceInterface/token）；稳定 AIDL 还给跨进程版本演化立规矩（第八章）。
- **为什么 system_server 31 线程且禁同步外呼？** 高并发预算 + 防优先级反转，框架进程的“特权”其实是“责任”（第十章）。
- **为什么事务缓冲区是事务即域？** 1MB 区是稀缺资源，驱动靠“用完即还”维持周转；Parcel 跨事务保留 = 机器里塞沙子（第六章）。

第十二章 深水区：Binder 观测武器库与 RPC 后端

12.1 三层观测（从粗到细）

层	手段	能看到什么
服务级	dumpsys binder_calls_stats（需开启）	每接口调用计数/耗时/拒绝码分布（BinderCallsStatsService，书1 附录B 曾列）
进程级	dumpsys binder <pid> / cat /d/binder/*	proc/thread 状态、事务栈、pending、缓冲水位（/d/binder/stats、transactions、proc/…）

层	手段	能看到什么
事务级	perfetto 的 binder_track	每次 BC_TRANSACTION→BR_REPLY 的时序轨（含目标服务/代码号），配合调度轨看唤醒等待

排障组合拳：卡顿先 `binder_calls_stats` 找热点接口 → `/d/binder/transactions` 看谁压栈未弹 → perfetto 精确到一次调用的唤醒-执行-回复三段。**事务栈里有别人=嵌套调用现场**（书3 §3.2 的 transaction_stack 实体就在这个 debugfs 文件里）。

12.2 RPC 后端（Android 13 的实验分支）

本树 `frameworks/native/libs/binder/RpcState.cpp` 提示了 Binder 的"第二人生"：**RPC backend**（`binder_rpc`）把事务协议搬到任意传输（Unix socket/TLS/USB）上运行，面向多设备场景（手机↔车机↔穿戴）——协议层 Binder 化、传输层可插拔。Parcel 因此有了"for RPC"分支（本册 transact :710 的 isForRpc 断言来源）。读码价值：看懂"Binder 是协议不是设备"的架构野心，也解释了 13 里 Parcel/状态码的一批防御性改动。

> **源码锚点 • 本章锚点**
> frameworks/native/libs/binder/Static.cpp：全进程 binder 统计与死亡通知簿
IPCThreadState.cpp 的 dump/命令字符串表（:75-97）
/d/binder/（debugfs，userdebug 可读）：stats/transactions/proc/<pid>/
RpcState.cpp：RPC 后端的事务簿记

附录 A 观测实战

目标	手段	对应机制
看服务名录	adb shell service list	servicemanager mNameToService
看某服务实现	dumpsys <名>（每个服务实现 dump）	服务端 onTransact 之外的 dump 通道
看线程池状态	debuggerd -b <pid> 看 binder 线程栈	joinThreadPool 循环
看调用统计	dumpsys binder_calls_stats（需开启）	BinderCallsStatsService（书1表）
验证身份切换	dumpsys activity 里 uid 记录	getCallingUid/clearCallingIdentity
复现 oneway 限制	大 oneway 事务 → FAILED_TRANSACTION	async space 配额
验证死亡通知	kill 某服务进程，观察名录自动除名	linkToDeath + BR_DEAD_BINDER

附录 B 三册联动：一条完整的因果链

第二册结尾 system_server 已就绪、Home 已启动。本册补完了此后每一天的运转逻辑：App 调用系统服务（本册第九章十跳）→ AMS 跨进程反向控制 App（IApplicationThread，同一机制的对称方向）→ 应用死亡由 AppDeathRecipient 清理（本册十章）→ 系统重启时名录与缓存全部重来（第二册启动链 + 本册句柄失效）。至此，“从开机到点击图标打开 App”的全栈白盒在你手中合拢。

附录 C 自检题

- 说出一次同步 Binder 调用的十跳（不看图）。
- `getCallingUid` 的值为什么可信？数据从哪来、经过哪几层？
- oneway 调用在什么时刻返回？什么情况下失败？失败码是什么？
- BinderProxy 与 BpBinder 的关系？为什么需要两层？
- AMS 为什么必须在某些操作前 clearCallingIdentity？举一个不这么做会出提权漏洞的场景。

附录 D 术语速查

术语	解释
Binder 驱动	/dev/binder 背后的内核模块，协议仲裁与一次拷贝的实现者
BC_* / BR_*	应用→驱动 / 驱动→应用 的命令前缀
binder_proc/thread/node/ref/buffer	驱动侧五类核心对象
transaction_stack	线程上的同步调用链栈，支撑跨进程递归
async_space / async_todo	oneway 专属配额与队列（保序）
BINDER_SET_CONTEXT_MGR	成为名录（句柄 0）的 ioctl
ProcessState / IPCThreadState	每进程 / 每线程 的 libbinder 单例
BBinder / BpBinder	native 服务端基类 / 远端代理（按句柄）
JavaBBinder / BinderProxy	Java Binder 的 native 壳 / native BpBinder 的 Java 马甲
Parcel（事务即域）	跨进程字节容器，生命周期随事务
flat_binder_object	Parcel 里"对象槽"的线格式（本地指针/远端句柄二态）
oneway	单向事务：COMPLETE 即返回，同目标保序，不保证执行
死亡通知	linkToDeath + BR_DEAD_BINDER 的对象死亡回调机制
Stability / VINTF	接口稳定性承诺等级与设备兼容矩阵

术语	解释
libbinder_ndk	面向 NDK/Rust 等的 C 稳定封装（AIBinder 家族）
三域	binder / hwbinder / vndbinder 的框架/硬件/厂商隔离

附录 E 动手实验（一天量）

实验一“名录巡礼”：`adb shell service list` 找到 activity/power/window；`adb shell service call activity 1`（code 需对照生成物，多半报错）——观察错误类型，理解 TRANSACTION code 的含义与 enforceInterface 的拦截。

实验二“线程构成”：对 system_server 连续抓三次 `debuggerd -b`（或 `cat /proc/<pid>/task/*/comm | grep Binder`），统计 binder 线程数变化，验证“31+1”预算与供给制扩池。

实验三“MiniBinder”：写一个 20 行的 App 内 demo——定义 `interface IDemo { int add(int a,int b); }` 的 aidl，Service 端实现并 addService（系统签名不可用时可用 bindService 路径替代名录），客户端调用。对照 8.3 骨架找到你代码里的 enforceInterface/transat 位置。

实验四“oneway 边界”：同一 aidl 方法加/去 oneway 各压测一百次，对比耗时分布；再传一个 2MB byte[]（oneway），复现 async 配额耗尽的 FAILED_TRANSACTION。

实验五“死亡通知”：bindService 后 kill 服务进程，客户端 DeathRecipient 回调链上打点，画出从进程死亡到你收到回调的时间线（对照 10.1）。

教科书习题集（分级）

第一级：简答题

1. Binder 的"一次拷贝"是怎么实现的？mmap 的是哪个进程的地址空间？

2. BC_TRANSACTION 和 BR_TRANSACTION 各是谁发给谁的？

3. BinderProxy 和 BpBinder 各在哪层？为什么需要两层？

4. oneway 的三个常见误区是什么？

5. servicemanager 为什么零线程池、仅 oneway？

6. Binder 的最大线程数是多少？system_server 是多少？为什么不同？

7. 事务即域（transaction-scoped）是什么意思？违反它会怎样？

8. linkToDeath 和 BR_DEAD_BINDER 的关系是什么？

第二级：分析题

9. **追踪时序**：画一次 oneway 调用的时序图，标出与同步调用的分岔点。

10. **身份链**：AMS 在处理一个 App 的 startActivity 时，Binder.getCallingUid() 经历了怎样的变化？什么时候该 clearCallingIdentity？

11. **数据结构推演**：如果 App A 和 App B 同时持有 "activity" 服务的 binder_ref，驱动侧有几个 binder_node？几个 binder_ref？画出关系图。

12. **失败场景**：system_server 崩溃后，正在等 BR_REPLY 的 App 会收到什么？已经注册的服务会发生什么？

第三级：设计题

13. **设计一个新 IPC**：假设 Binder 不存在，你会怎么设计 Android 的 IPC？从安全（身份）、性能（拷贝）、能力传递三个维度给出方案，并对比你的方案与 Binder 的优劣。

14. **调试实战**：用户报告"App 调用系统服务偶尔超时"，给你一台 userdebug 设备，设计一套完整定位方案（从 dumpsys 到 perfetto 到代码审查）。

参考答案要点

1. 接收方 mmap 自己的地址空间（1MB-2页只读映射），驱动 copy_from_user 直写接收方的映射区。

6. 普通进程 15+1=16；system_server 31+1=32（SystemServer.java:435）——框架进程并发需求高。

10. 收到事务时 getCallingUid=App uid；AMS 代行权限前 clearCallingIdentity 恢复 system 身份。

12. 等 reply 的 App 收到 BR_DEAD_REPLY（DEAD_OBJECT 异常）；servicemanager 的 linkToDeath 触发自动除名。

术语表（按拼音排序）

术语	所在册	解释
AIDL	3	Android Interface Definition Language，自动生成 Stub/Proxy
async space	3	oneway 事务的独立配额（约映射区一半）
BBinder	3	Native 服务端基类（onTransact 入口）

术语	所在册	解释
BC_*	3	应用→驱动的命令前缀（BC_TRANSACTION 等）
Binder	3	Android 核心 IPC：一次拷贝+内核身份+能力传递
BinderProxy	3	Java 层的远端代理（BpBinder 的马甲）
BpBinder	3	Native 层的远端代理（持 32 位句柄）
BR_*	3	驱动→应用的命令前缀（BR_TRANSACTION 等）
binder_proc	3	驱动侧的进程对象（threads/nodes/refs 树）
binder_ref	3	驱动侧的客户端引用（desc=句柄）
binder_node	3	驱动侧的服务端点（ptr/cookie 指向 BBinder）
binder_thread	3	驱动侧的线程对象（transaction_stack 栈）
flat_binder_object	3	Parcel 内的对象槽线格式（本地指针/远端句柄二态）
FROZEN_REPLY	3	12+ 冻结进程的同步调用快速失败
handle 0	3	servicemanager 的固定句柄（自举）
IPCThreadState	3	每线程的命令收发中枢（mIn/mOut 缓冲）
oneway	3	单向事务：BR_TRANSACTION_COMPLETE 即返回
Parcel	3	跨进程字节容器（事务即域）
ProcessState	3	每进程的 Binder 资产（open/mmap/线程池）
RPC backend	3	13 实验分支：Binder 协议跑在任意传输上
servicemanager	3	Binder 服务名录（零线程池+仅 oneway）
Stub/Proxy	3	AIDL 生成的服务端存根/客户端代理
transaction_stack	3	线程上的同步调用链栈（支持跨进程递归）
事务即域	3	Parcel 生命周期随事务——跨事务保留=使用已回收内存

排障手册：Binder 问题定位

跨进程调用卡住

1. dumpsys binder 查目标进程的线程状态（available/max）
2. debuggerd -b 看目标进程 binder 栈（谁在忙）
3. perfetto binder_track 看唤醒-执行段耗时

→ 本册第五/十章

服务注册失败

1. service list 确认名字
2. 检查 SELinux（logcat -b events | grep avc）
3. dumpsys servicemanager 看注册日志

→ 本册第七章 + 书1 § 十六

oneway 大事务失败

1. 异常码 FAILED_TRANSACTION
2. 检查数据大小（>500KB 可能超 async space）
3. 改用 fd+SharedMemory 或分片

→ 本册第三/十章

BinderProxy 泄漏

1. logcat 搜 "many live BinderProxies"
2. dumpsys binder 的 proxy 计数
3. 检查 linkToDeath/unlinkToDeath 是否成对

→ 本册第八/十章

system_server 崩溃后 App 恢复

1. 理解 runtime restart 场景（本册第九章）
2. App 需处理 DEAD_OBJECT 重试逻辑

3. ServiceManager 缓存会自动失效

→ 本册第九章